

# spice\_discontinuity algorithm running-time study (theoretical)

## Scope and notation

This study covers algorithm-bearing functions in:

- `spice_discontinuity/find.py`
- `spice_discontinuity/inject.py`
- `spice_discontinuity/generate.py`

Symbols used below:

- $N$ : number of samples in a 1D series (`x`, `y`, or `values`)
- $K$ : number of injected random spikes ( $K \leq N$ )
- $R$ : number of CSV rows
- $M$ : number of CSV columns
- $C$ : number of polynomial coefficients
- $D$ : number of DataFrame columns

Complexities are asymptotic model-level estimates (Big-O), not wall-clock guarantees.

## Complexity summary

| Module               | Function                                    | Time complexity                 | Notes                                                                                         |
|----------------------|---------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------------------|
| <code>find.py</code> | <code>load_csv_numeric_columns(path)</code> | $O(R * M)$                      | Visits every cell once (parse/skip), builds numeric-column lists.                             |
| <code>find.py</code> | <code>_mad(values)</code>                   | $O(N \log N)$<br>(conservative) | Dominated by two median computations; many NumPy builds are near-linear in practice.          |
| <code>find.py</code> | <code>score_series(x, y)</code>             | $O(N \log N)$<br>(conservative) | Mostly linear diff/divide passes; <code>_mad</code> dominates under conservative median cost. |

| Module      | Function                                              | Time complexity                 | Notes                                                                                                         |
|-------------|-------------------------------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| find.py     | <code>detect_robust(x, y, ...)</code>                 | $O(N \log N)$<br>(conservative) | <code>score_series</code> +<br>finite-mask passes<br>+ <code>find_peaks</code><br>(typically<br>near-linear). |
| find.py     | <code>detect(x, y, ...)</code>                        | $O(N \log N)$<br>(conservative) | Thin wrapper<br>around<br><code>detect_robust</code> .                                                        |
| inject.py   | <code>inject_step(values, index, magnitude)</code>    | $O(N)$                          | List copy is $O(N)$ ;<br>tail update loop<br>is $O(N - \text{index})$ .                                       |
| inject.py   | <code>inject_spike(values, index, magnitude)</code>   | $O(N)$                          | List copy<br>dominates; one<br>index update is<br>$O(1)$ .                                                    |
| inject.py   | <code>inject_random_spikes(values, count, ...)</code> | $O(N + K)$                      | Copy + random<br>sampling + $K$<br>updates; worst<br>case with $K \sim N$<br>is $O(N)$ .                      |
| inject.py   | <code>inject_faults(df, fault_percentage)</code>      | $O(N * D + F)$                  | $F$ fault points,<br>but full<br>DataFrame copy<br>drives cost ( $D$<br>columns).                             |
| generate.py | <code>polynomial(n, coefficients, ...)</code>         | $O(N * C)$                      | <code>np.polyval</code> over<br>$N$ samples and<br>polynomial<br>degree/coeff<br>count $C$ .                  |
| generate.py | <code>sinusoid(n, ...)</code>                         | $O(N)$                          | <code>linspace</code> +<br>elementwise <code>sin</code> .                                                     |
| generate.py | <code>exponential(n, ...)</code>                      | $O(N)$                          | <code>linspace</code> +<br>elementwise <code>exp</code> .                                                     |
| generate.py | <code>to_csv(path, x, y, ...)</code>                  | $O(N)$                          | Row-wise write<br>loop over zipped<br>arrays<br>(I/O-bound in<br>practice).                                   |

$F$  in `inject_faults` is the number of injected faults (`max(1, floor(N * fault_percentage))`).

## Derivation notes by module

### `find.py`

1. `score_series` performs a sequence of first-order and second-order finite-difference passes and normalizations over arrays whose lengths are all linear in  $N$ .
2. `_mad` computes medians (central location and absolute deviation median). Under a conservative model where median requires sorting/partition work that scales superlinearly, this contributes  $O(N \log N)$  and dominates.
3. `detect_robust` adds finite checks, masking, and `scipy.signal.find_peaks`; these are generally linear scans relative to score length, so the same dominant bound from `score_series` carries through.

### `inject.py`

1. All list-based injectors start with `output = list(values)`, which is  $O(N)$ .
2. `inject_step` then updates a suffix ( $N$ -index elements), while `inject_spike` updates exactly one element.
3. `inject_random_spikes` adds random index selection and  $K$  point updates; this is naturally expressed as  $O(N + K)$ .
4. `inject_faults` is DataFrame-oriented: full `df.copy()` cost scales with both rows and columns, then the algorithm modifies one copied NumPy column and assigns it back.

### `generate.py`

1. `polynomial` complexity depends on sample count and polynomial size: evaluating a degree- $C-1$  polynomial at  $N$  points is  $O(N * C)$ .
2. `sinusoid` and `exponential` are vectorized elementwise transforms over  $N$  points ( $O(N)$  each).
3. `to_csv` is a single pass through paired arrays; asymptotically linear in rows written.

## Best practical ways to test performance in this project

1. **Benchmark each core path separately:** benchmark `score_series`, `detect_robust`, each injector, and generators independently before benchmarking full CLI workflows.
2. **Run scaling sweeps:** test geometric  $N$  ranges (for example `1e3`, `1e4`, `1e5`, `1e6`) and, where relevant, vary  $K$ ,  $C$ , and `fault_percentage`.
3. **Use deterministic inputs:** fix seeds for random injection and use reproducible synthetic datasets so trend changes are attributable to code changes.
4. **Reduce measurement noise:** isolate CPU-heavy benchmarks from plotting/file I/O where possible, run multiple processes/iterations, and compare medians plus spread (not single runs).

5. **Track scaling shape, not just raw time:** confirm observed growth aligns with expected complexity classes ( $O(N)$ ,  $O(N \log N)$ ,  $O(N * C)$ ).

## Python benchmarking tool recommendation

For a project like this, **pyperf** is the best primary choice and the closest Python analogue to “standard benchmark harness” usage in C++ ecosystems:

- It is designed for reliable Python benchmarking (process control, warmups, repeated runs, statistics).
- It is better suited than ad-hoc **timeit** snippets for comparing algorithm implementations over time.
- It fits this repository well because the hot paths are numerical array computations where measurement stability matters.

Practical positioning:

- Use **pyperf** for authoritative benchmark scripts/results.
- Use **timeit** for quick local micro-checks.
- Use **pytest-benchmark** only if you want benchmark assertions integrated into a pytest-based test workflow (this repo currently uses **unittest**).